

Taller Práctico: Asincronía y Callbacks en JavaScript

Objetivo

Implementar y analizar soluciones utilizando callbacks y tareas asíncronas en JavaScript para comprender cómo funciona el comportamiento no bloqueante del lenguaje.

Parte 1. Callbacks Básicos

Un callback es una función que se envía como argumento a otra función para ser ejecutada posteriormente.

Analiza el siguiente código y completa las secciones marcadas con TODO.

```
function realizarOperacion(num1, num2, operacionCallback) {
    console.log(`Operación con: ${num1} y ${num2}`);
    operacionCallback(num1, num2);
}

function sumar(a, b) {
    console.log(`Resultado Suma: ${a + b}`);
}

// TODO
function restar(a, b) {

}

// TODO
function multiplicar(a, b) {

}

realizarOperacion(10, 5, sumar);

// TODO
// Implementar las tres operaciones.
```

Actividades

1. Implementa el callback `restar`.
2. Implementa el callback `multiplicar`.
3. Ejecuta las tres operaciones.
4. Captura la salida obtenida en consola.

```
> function realizarOperacion(num1, num2, operacionCallback) {  
  console.log(`Operación con: ${num1} y ${num2}`);  
  operacionCallback(num1, num2);  
}  
  
function sumar(a, b) {  
  console.log(`Resultado Suma: ${a + b}`);  
}  
  
function restar(a, b) {  
  console.log(`Resultado resta: ${a - b}`);  
}  
  
function multiplicar(a, b) {  
  console.log(`Resultado Suma: ${a * b}`);  
}  
  
realizarOperacion(10, 5, sumar);  
realizarOperacion(10, 5, restar);  
realizarOperacion(10, 5, multiplicar);  
  
Operación con: 10 y 5  
Resultado Suma: 15  
Operación con: 10 y 5  
Resultado resta: 5  
Operación con: 10 y 5  
Resultado Suma: 50
```

Preguntas de reflexión

1. ¿Qué ventaja tiene enviar una función como parámetro?

La ventaja que tiene esto es que se vuelve un poco más flexible

2. ¿Qué ocurre si enviamos una función diferente como callback?

Simplemente cambia el resultado el código se adapta a lo que se le pida.

3. ¿La función principal necesita conocer qué operación matemática se ejecutará?

No porque la función principal solo se encarga de ejecutar lo que se le mandes mas no tiene que saber si es suma, resta o multiplicación eso lo decide el callback

Parte 2. Comprendiendo la Asincronía

Observa cuidadosamente el siguiente programa.

```
function tareaNoBloqueante(callback) {  
    console.log("Iniciando tarea no bloqueante...");  
  
    setTimeout(function() {  
        console.log("Tarea completada.");  
        callback();  
    }, 2000);  
}  
  
console.log("Inicio del programa.");  
  
tareaNoBloqueante(function() {  
    console.log("Continuando después de la tarea.");  
});  
  
console.log("Fin del programa.");
```

Actividad

Antes de ejecutar el programa:

1. Escribe el orden que crees que aparecerá en la consola.
2. Ejecuta el código.
3. Registra el resultado real.

```

> function tareaNoBloqueante(callback) {
    console.log("Iniciando tarea no bloqueante...");

    setTimeout(function() {
        console.log("Tarea completada.");
        callback();
    }, 2000);
}

console.log("Inicio del programa.");

tareaNoBloqueante(function() {
    console.log("Continuando después de la tarea.");
});

console.log("Fin del programa.");

```

Inicio del programa.
 Iniciando tarea no bloqueante...
 Fin del programa.
 < undefined
 Tarea completada.
 Continuando después de la tarea.

Análisis

Responde:

orden mio:

- Inicio del programa.
- Iniciando tarea no bloqueante...
- Tarea completada.
- Continuando después de la tarea.
- Fin del programa.

1. ¿Coincidió tu predicción con el resultado?

Casi pero no, uno pensaría que el programa iba a esperar a que terminara la tarea antes de seguir pero no el mensaje de "Fin del programa" salió antes que "Tarea completada"

2. ¿Por qué el mensaje "Fin del programa" aparece antes que "Tarea completada"?

Porque `setTimeout` no detiene el programa. Lo que hace es decir "oye, dentro de 2 segundos ejecuta esto" y mientras tanto el HP código sigue corriendo como si nada

3. ¿Qué papel cumple `setTimeout()` en este comportamiento?

bloquea el hilo principal agenda la tarea para más tarde

Parte 3. Simulación de Procesos Reales

Debes simular tres tareas comunes de una aplicación moderna.

Código Base

```
function solicitarJSON(callback) {
  setTimeout(() => {
    console.log("Tarea: solicitar JSON completada.");
    callback();
  }, 3000);
}

function reproducirAudio(callback) {
  setTimeout(() => {
    console.log("Tarea: audio reproducido.");
    callback();
  }, 1000);
}

function leerSensor(callback) {
  setTimeout(() => {
    console.log("Tarea: lectura de sensor completada.");
    callback();
  }, 500);
}

console.log("--- Iniciando procesos asíncronos ---");

solicitarJSON(() => console.log("Archivo JSON recibido."));
```

```
reproducirAudio(() => console.log("Audio finalizado.));  
leerSensor(() => console.log("Datos del sensor listos.));
```

Requisitos

Implementa cada función utilizando `setTimeout()`.

Función	Tiempo
solicitarJSON	3000 ms
reproducirAudio	1000 ms
leerSensor	500 ms

Actividades

1. Implementa las tres funciones.
2. Ejecuta el programa.
3. Registra el orden real de finalización.

Resultado

Orden	Tarea
1	lectura de sensor completada.
2	audio reproducido.
3	solicitar JSON completada.

Preguntas

1. ¿Cuál terminó primero?

La tarea de leerSensor, porque tenía el tiempo más corto (500 ms).

2. ¿Cuál terminó último?

La tarea de solicitarJSON, ya que se programó con el mayor tiempo (3000 ms).

3. ¿Por qué el orden de finalización es diferente al orden de ejecución?

Porque aunque las funciones se ejecutan en orden como esta en el ejemplo: solicitarJSON, luego reproducirAudio, luego leerSensor, pero al agregarle

setTimeout() se les agenda un tiempo distinto a cada función donde determina el orden finalización cambiando el orden de ejecución que lleva el código.

Parte 4. Diseñando tu Propia Operación Asíncrona

Reto

Crea una función llamada:

simularDescarga(nombreArchivo, callback)

La función deberá:

1. Mostrar un mensaje indicando que inició la descarga.
2. Esperar 4 segundos utilizando `setTimeout()`.
3. Mostrar un mensaje indicando que la descarga terminó.
4. Ejecutar el callback recibido.

Ejemplo esperado

```
simularDescarga("manual.pdf", () => {  
    console.log("Archivo listo para abrir.");  
});
```

Evidencia

```
> function simularDescarga(nombreArchivo, callback) {  
    console.log(`📄 Iniciando descarga de ${nombreArchivo}...`);  
  
    setTimeout(() => {  
        console.log(`✅ Descarga de ${nombreArchivo} completada.`);  
        callback();  
    }, 4000);  
}  
  
simularDescarga("manual.pdf", () => {  
    console.log("📁 Archivo listo para abrir.");  
});  
📄 Iniciando descarga de manual.pdf...  
< undefined  
✅ Descarga de manual.pdf completada.  
📁 Archivo listo para abrir.
```

Parte 5. Análisis del Callback Hell

Observa la siguiente estructura:

```
obtenerUsuario(() => {  
  obtenerPedidos(() => {  
    obtenerFactura(() => {  
      enviarCorreo(() => {  
        console.log("Proceso finalizado");  
      });  
    });  
  });  
});
```

Actividades

1. Describe el problema visual que presenta el código.

El código se ve como una pirámide hacia la derecha, Cada función depende de la anterior y se anida dentro de otra y esto genera un efecto escalonado que dificulta la lectura y comprensión.

2. Explica qué es el Callback Hell.

El Callback Hell es ese “infierno” que ocurre cuando dependes de demasiados callbacks uno dentro de otro

3. ¿Por qué dificulta el mantenimiento de aplicaciones grandes?

cuesta seguir el flujo porque todo está anidado, si quieres cambiar un paso, puedes romper toda la cadena, depurar es un dolor de cabeza porque no sabes en qué nivel falló, en proyectos grandes, este estilo se vuelve insostenible e Investiga y menciona dos alternativas modernas para evitar este problema.

Entregables

El estudiante deberá entregar:

- Código fuente de todos los ejercicios.
- Capturas de consola de las ejecuciones.
- Respuestas a las preguntas de análisis.
- Explicación del ejercicio de Callback Hell.